

Optimisation du temps d'exécution des requêtes SQL

Eliott PAQUET N° 30960

2025 – 2026

Sommaire

Les bases du traitement des requêtes SQL

Implémentation d'un Algebrizer en c++

La gestion mémoire

Optimisation du plan

Optimisation dépendante des données

analyse des résultats

Conclusion

Chiffres clés sur l'utilisation de SQL

► Amazon Aurora

- Jusqu'à **500 000 lectures/sec** et **100 000 écritures/sec**
- Soit environ **600 000 requêtes SQL par seconde**
- $\approx$ **51 milliards de requêtes par jour**
- Source : <https://aws.amazon.com/blogs/aws/now-available-amazon-aurora/>

► Oracle Database (Exadata)

- Jusqu'à **1 million de transactions par seconde**
- $\approx$ **86 milliards de requêtes par jour**
- Source : benchmark Exadata (IT Convergence)

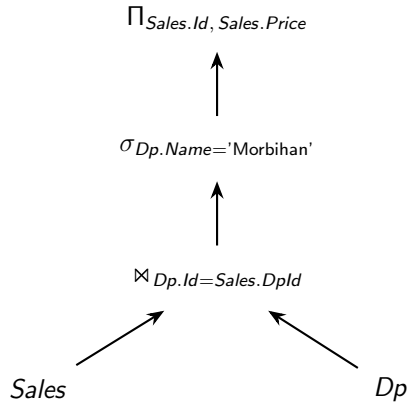
Edgar F. Codd (1970/1971)

Calcul relationnel $\iff$ Algèbre relationnelle

```

1 SELECT S.Id, S.Price
2 FROM Sales
3 JOIN Dp ON Dp.Id = S.DpId
4 WHERE Dp.Name = 'Morbihan';

```



Problématique

Comment créer, optimiser et exécuter un plan de requête ?

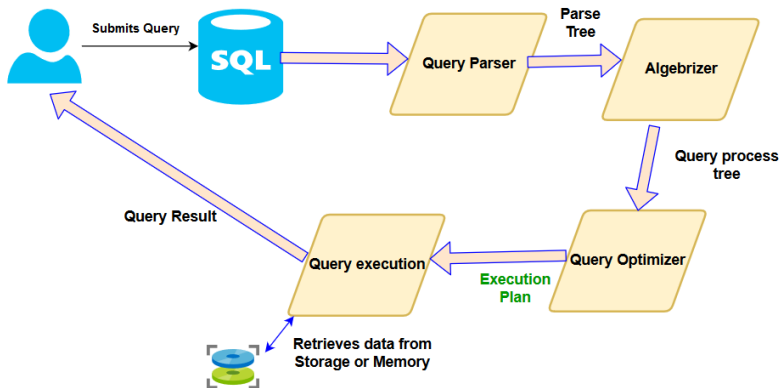


Figure: Traitement d'une requête SQL par un SGBD.

Les informations disponibles à l'entrée de l'algebrizer

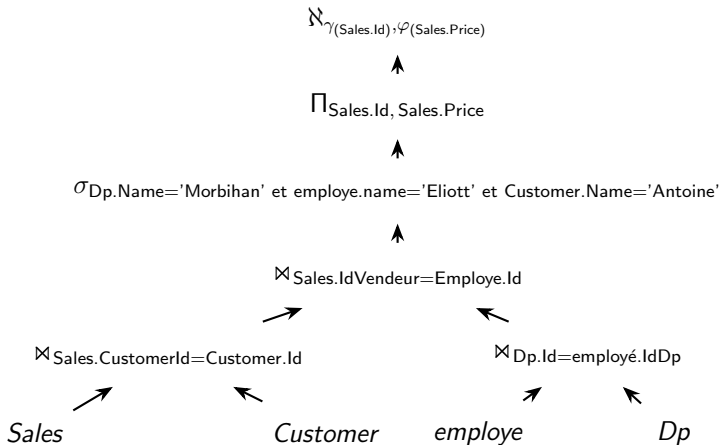
► Parser

- Colonne sélectionné
- Différente Jointures
- Condition
- Group By /Agrégation

► Mémoire

- différentes Tables/Colonnes
- Nombre de ligne
- autres statistiques (tri,indexation,cracking)

Arbre Naïf



Les bases de données orientées colonnes

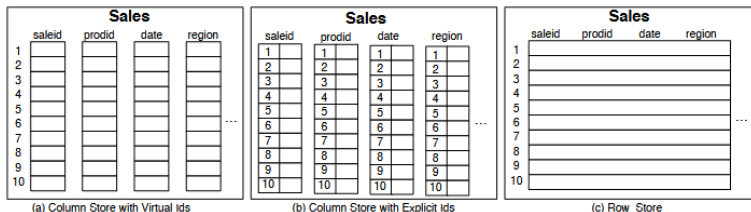
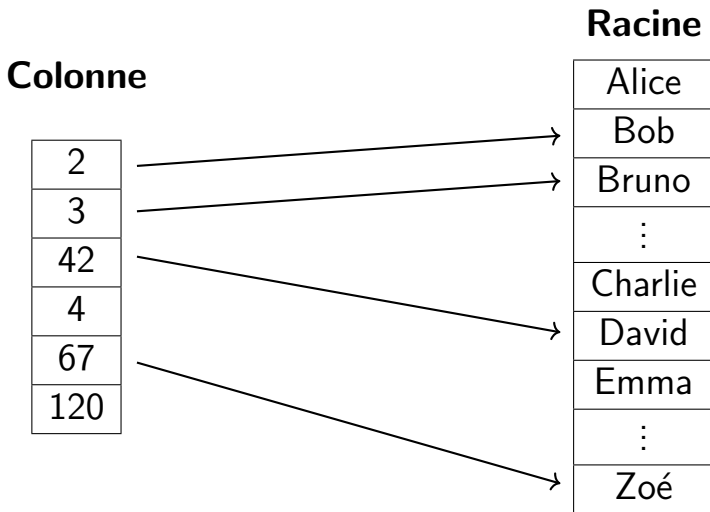
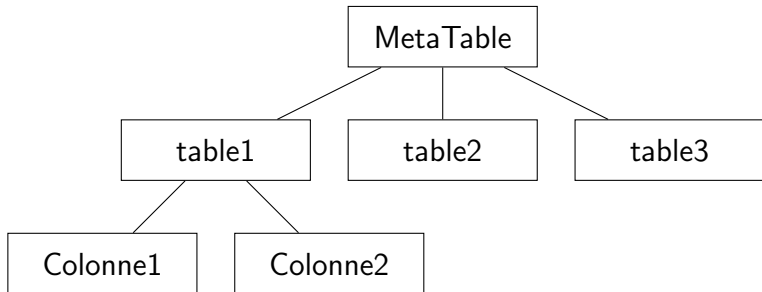


Figure: Rendu physique des BDD orientées colonnes et BDD orientées lignes

Optimisation mémoire



Notre Premier Modèle



- ▶ **Colonne** : Racine et liste de position dans la racine encore valide
- ▶ **Table** : liste de Colonne encore valide
- ▶ **MetaTable** : Liste de plusieurs *Tables*.
- ▶ **Particularité** : à une même position, les valeurs des colonnes forment une ligne de donnée

Application sur nos opérations

Selection

2
3
42
35
12
98
67
420

 $\Rightarrow$

2
67
420

Jointure

2
3
42
35
12
98
67
420

 $\bowtie$

54
78
904

 $\Rightarrow$

2	78
3	54
3	78
3	904
98	78
98	904
420	54

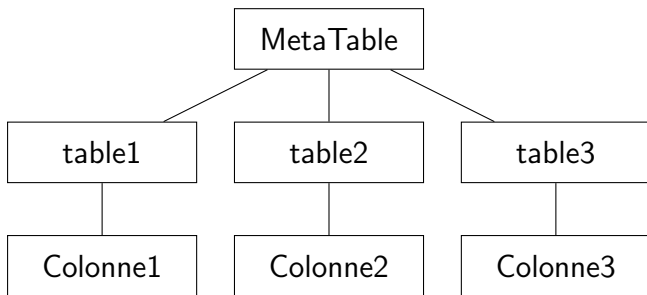
Invariant lors de l'exécution

Les lignes formé par la concaténation de toutes les colonnes d'une Meta-Table restent les mêmes jusqu'à la fin

Les lignes formé par la concaténation de toutes les colonne d'une Table sont les même à l'initialisation

Les Colonnes d'une même table sont des doublons !

Optimisation de notre modèle



- **Particularité** : Une colonne par table (ne dépend pas du nombre de colonnes)
- **Résultat** : 45 min → 22 secondes

Quelques Optimisation de l'arbre Naïf

- ▶ Descente de sélection
- ▶ Ajout de projection

Graphe présentant les 3 versions en fonction de la taille du jeux de donnée

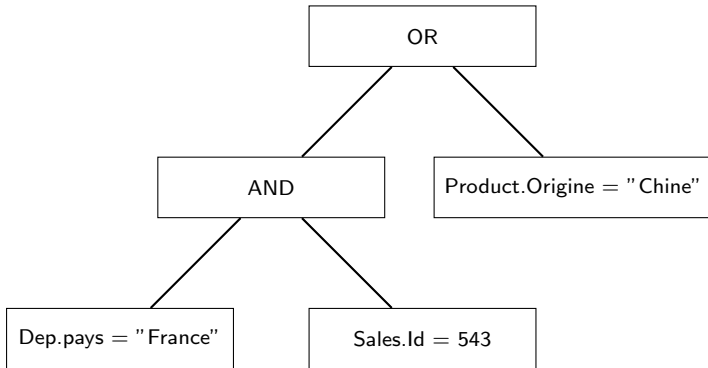
Les différents types de jointures possibles

- ▶ Produit Cartésien : $\mathcal{O}(n \times m)$
- ▶ Jointure Triée : $\mathcal{O}(n \log(n) + n + m \log(m) + m)$
- ▶ Jointure par dictionnaire : $\mathcal{O}(2n)$

graphe présentant les 3 type de jointure en fonction du nombre de données

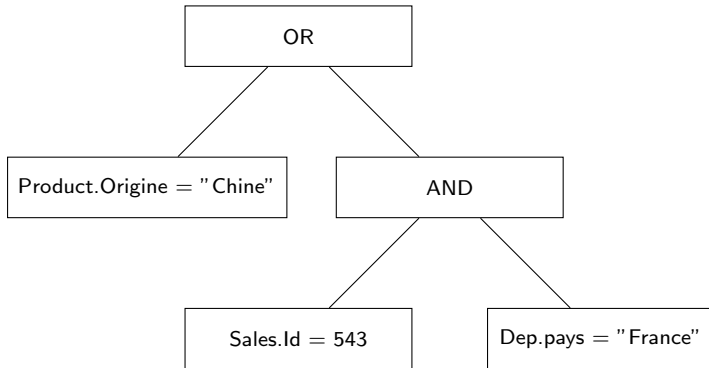
Analyse des expressions binaires

```
1 WHERE (Dep.pays = "France" AND Sales.Id = 543) OR (Product.  
    Origine = "Chine")
```



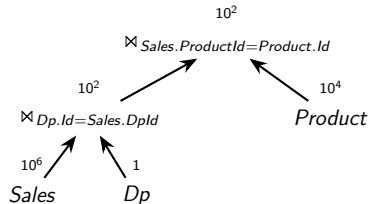
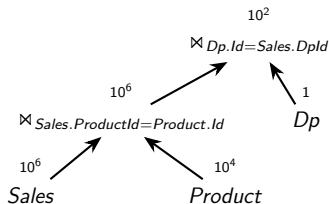
Optimisation des expressions binaires

```
1 WHERE (Dep.pays = "France" AND Sales.Id = 543) OR (Product.  
    Origine = "Chine")
```



graphe opti binaire

Résultats équivalents mais complexité différente



- ▶ $\bowtie_{Sales.ProductId=Product.Id}$ d'abord $\rightarrow 10^4 \times 10^6 + 10^6 \times 1$ comparaisons
- ▶ $\bowtie_{Dp.Id=Sales.DpId}$ d'abord $\rightarrow 1 \times 10^6 + 10^2 \times 10^4$ comparaisons

Comment choisir l'ordre ?

Mon Algorithme

BUT : Minimiser à tout moment le nombre de ligne

Def : Rapport de Cardinalité $\rightarrow \frac{\text{Estimation du nombre de ligne créé}}{\text{produit du nombre de ligne testé}}$

1. Calcul le RC de chaque Jointure
2. Trie les Jointure par RC croissant
3. Ajoute les jointures une par une à l'aide de notre structure Union-Find sur les tables

graphe ordre de jointure

Analyse de toutes les optimisations

Graphe toute optimisation

Résumé

- ▶ Création d'un Algebrizer pour transformer le SQL en des opérations exécutables
- ▶ Optimisations de notre modèle de représentation mémoire
- ▶ Optimisation naïve de l'arbre naïf
- ▶ Optimisation dépendantes des données

Ouverture

Les pistes récentes :

- ▶ Analyse des données $\Rightarrow$ Machine Learning (Génération de plan, Choix de l'ordre des jointures, ...)
- ▶ Une meilleure organisation des données en mémoire (Voire le travail d'Antoine)
- ▶ Les deux: *Hypothetical Index Benefit Estimation on Column-oriented Databases using Quantiles* 2025

Nos ressources

Nos ressources :

- ▶ *The Design and Implementation of Modern Column-Oriented Database Systems* 2013
- ▶ *C-Store: A Column-oriented DBMS* 2005
- ▶ *A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration* 2021
- ▶ *Extensible Query Optimizers in Practice* 2024
- ▶ *Foundation of Databases* 1995